

Hierarchical Abstract Tree for Cross-Document Retrieval-Augmented Generation using NVIDIA CUDA-Accelerated GPU Computing

Nivesh Jain

(Dept. of Computer Engineering, Vishwakarma Institute of Technology, Pune, India
nivesh.jain24@vit.edu)

Abstract: Retrieval-augmented generation (RAG) connects language models with external evidence, but conventional flat chunk retrieval is not well matched to questions whose evidence is distributed across several documents or expressed at different levels of abstraction. This paper proposes a Hierarchical Abstract Tree (HAT), a document representation in which source passages are embedded as leaf nodes and recursively grouped into higher-level semantic abstractions. The hierarchy is used as a coarse-to-fine retrieval structure: broad nodes identify relevant regions of the corpus, while leaf nodes preserve the source passages required for evidence-grounded generation. The proposed design is explicitly aligned with NVIDIA CUDA-accelerated GPU computing, assigning dense embedding operations, batched similarity computation, and candidate selection to data-parallel GPU execution while retaining provenance and document-level routing information. The paper develops the HAT architecture, retrieval formulation, CUDA-oriented execution model, complexity analysis, and an evaluation protocol for cross-document question answering. The paper also defines a reproducible experimental methodology for evaluating retrieval quality, answer grounding, and GPU execution efficiency.

Keywords: retrieval-augmented generation, hierarchical retrieval, cross-document question answering, semantic abstraction, vector search, CUDA, GPU computing, document retrieval, evidence grounding

Categories: H.3.1, H.3.3, H.3.4, H.3.7, I.2.7

DOI: 10.3897/jucs.<SubmissionNumber>

1 Introduction

Large language models (LLMs) provide strong capabilities for question answering, summarization, and information synthesis, but their parametric knowledge is static with respect to the model’s training process and does not by itself provide reliable source provenance. Retrieval-augmented generation (RAG) addresses this limitation by retrieving external passages and conditioning generation on the retrieved evidence [Lewis et al. 2020]. Earlier systems such as REALM demonstrated retrieval as an explicit interface to external knowledge [Guu et al. 2020], while dense passage retrieval (DPR) showed that learned vector representations can support effective open-domain retrieval [Karpukhin et al. 2020].

The dominant practical RAG pattern is comparatively flat: documents are divided into chunks, each chunk is embedded, a query is embedded, and the top- k chunks are retrieved using a vector similarity index. This design is simple and effective for questions whose evidence is local. It is less natural for cross-document

questions in which the answer requires a combination of facts, concepts, or relationships distributed across several sources. In such cases, retrieval must perform two different tasks: navigation, which identifies the relevant semantic region of the collection, and evidence selection, which identifies the passages that directly support the answer.

Increasing the generator context window does not eliminate this problem. Long-context studies show that language models may use relevant information unevenly depending on its position in the supplied context, including substantial degradation when relevant material appears in the middle of a long input [Liu et al. 2024]. Architectures such as Longformer reduce the cost of processing long sequences, but they do not replace the need to select a compact, relevant evidence set before generation [Beltagy et al. 2020]. Consequently, retrieval structure remains an important component of long-document and multi-document question answering.

Hierarchical retrieval offers a promising alternative. RAPTOR recursively embeds, clusters, and summarizes text into a tree so that retrieval can occur at different semantic resolutions [Sarathi et al. 2024]. HAT builds on this principle but focuses specifically on cross-document retrieval and on the systems problem of executing large portions of the retrieval pipeline efficiently on NVIDIA GPUs. The proposed hierarchy is not intended to replace leaf evidence with summaries; rather, summaries serve as routing representations while the final answer is grounded in source-level nodes.

The contributions of this paper are fourfold: (1) a hierarchical document representation that preserves provenance while introducing multi-level semantic abstractions; (2) a coarse-to-fine cross-document retrieval procedure; (3) a CUDA-oriented execution model for embedding, similarity, and candidate-selection stages; and (4) a reproducible evaluation protocol that distinguishes retrieval quality, answer quality, and systems efficiency. The work is presented as a proposed architecture and research methodology; no unsupported experimental result is reported.

2 Related Work

2.1 Retrieval-Augmented Generation and Dense Retrieval

RAG combines a generator with a non-parametric retriever so that model outputs can be conditioned on externally stored information [Lewis et al. 2020]. REALM introduced retrieval as an explicit interface to external knowledge during language-model pre-training [Guu et al. 2020], while DPR established a dual-encoder formulation for dense passage retrieval [Karpukhin et al. 2020]. ColBERT explored late interaction for scalable passage search [Khattab and Zaharia 2020]. These methods provide the retrieval primitives used by HAT, but they do not define a provenance-preserving hierarchy whose internal representations are explicitly used for cross-document navigation.

2.2 Hierarchical and Long-Context Retrieval

Long-document work includes Longformer and Fusion-in-Decoder [Beltagy et al. 2020, Izacard and Grave 2021]. *Lost in the Middle* shows that longer context

does not guarantee uniform use of evidence [Liu et al. 2024], while recent comparison of RAG and long-context models highlights a cost-accuracy trade-off [Li et al. 2024]. RAPTOR is the closest predecessor: it recursively embeds, clusters, and summarizes text into a retrieval tree [Sarathi et al. 2024]. HAT shares this multi-resolution idea but targets cross-document evidence discovery, explicit provenance, and CUDA-oriented retrieval execution.

2.3 Vector Indexing and GPU Computing

HNSW reduces approximate-nearest-neighbor search cost through hierarchical graph organization [Malkov and Yashunin 2020]. GPU similarity search demonstrates efficient parallel distance computation and top- k selection [Johnson et al. 2017], while CUDA provides the execution and memory model for mapping dense numerical stages to GPU kernels [NVIDIA 2026]. IO-aware GPU design further motivates minimizing memory movement and synchronization [Dao et al. 2022]. These approaches accelerate vector operations but do not themselves define semantic abstraction trees or provenance-aware cross-document evidence composition.

3 Problem Formulation and Design Goals

Let a corpus be $\mathcal{D} = \{d_1, d_2, \dots, d_M\}$. Each document is segmented into semantically coherent leaf passages $p_{i,j}$, where i identifies the source document and j identifies a passage within that document. An encoder $f(\cdot)$ maps a passage or abstraction to an embedding in $\mathbb{R}^d$. For a query q , the base similarity function is cosine similarity,

$$s(q, x) = \frac{f(q) \cdot f(x)}{\|f(q)\|_2 \|f(x)\|_2}. \quad (1)$$

A flat retriever searches all leaf embeddings. HAT instead constructs a rooted tree $T = (V, E)$ in which leaves correspond to source passages and internal nodes represent abstractions over groups of child nodes. Each node v stores an embedding z_v , an abstraction text a_v when applicable, and provenance metadata P_v identifying descendant documents and passages. The retrieval objective is to return a compact evidence set E_q that maximizes relevance while preserving source diversity and traceability.

The design follows four goals. First, **semantic hierarchy**: the structure should represent broad and fine-grained concepts simultaneously. Second, **evidence preservation**: internal summaries must not be treated as substitutes for source passages during final grounding. Third, **cross-document routing**: internal nodes may group evidence from multiple documents when they express a shared topic or relation. Fourth, **GPU suitability**: the dominant numerical operations should be expressed as batched, parallel computations with limited host-device synchronization.

These goals target a gap between flat dense RAG, semantic tree retrieval, and GPU vector search: HAT treats the hierarchy simultaneously as a cross-document semantic router, a provenance-preserving evidence structure, and a workload decomposition mechanism.

3.1 Testable Novelty Claims

Four claims guide validation: **C1**, hierarchical routing should recover multi-source evidence with fewer leaf candidates at comparable recall; **C2**, provenance-preserving abstractions should improve evidence traceability; **C3**, cross-document grouping should improve multi-source recall over document-local trees; and **C4**, GPU-resident similarity and top- k selection should improve retrieval throughput for sufficiently large workloads. If controlled experiments do not support these claims, the proposed architecture should not be considered superior to the baselines.

4 Proposed Hierarchical Abstract Tree

4.1 Tree Construction

The construction process begins by segmenting each source document into leaf passages. Each passage is embedded and stored with its document identifier, position, and optional metadata. Semantically related nodes are then grouped to form the next level. For each group, an abstraction is generated that captures the common information represented by its children, and the abstraction is embedded into the same vector space. The process repeats until a compact root or small set of upper-level nodes is obtained.

Unlike a purely document-local hierarchy, HAT permits grouping across document boundaries. A group can therefore represent a theme such as a common method, event, entity, or technical concept occurring in several sources. Provenance is retained through descendant pointers, so every abstraction can be expanded back to its supporting passages.

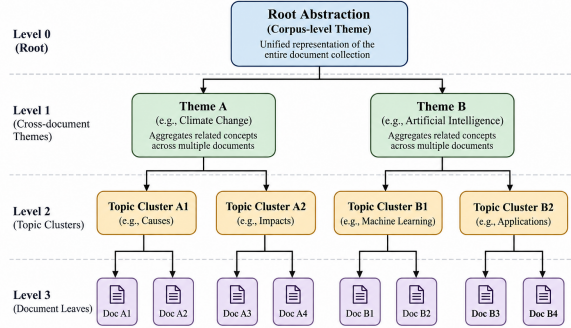


Figure 1: Hierarchical Abstract Tree for Cross-Document Retrieval

Figure 1: Hierarchical Abstract Tree for cross-document retrieval. Internal nodes represent increasingly broad semantic abstractions, while document leaves preserve source-level evidence and provenance.

The hierarchy can be represented by a node tuple

$$v = (z_v, a_v, P_v, c_v), \quad (2)$$

where z_v is the embedding, a_v is the abstraction text, P_v is provenance, and c_v is the set of child nodes. The leaf case contains the original passage instead of a generated abstraction. This explicit separation supports citation-aware generation and allows the hierarchy to be updated without losing source references.

4.2 Coarse-to-Fine Retrieval

At query time, the query is embedded once and compared with upper-level node embeddings. The highest-scoring nodes are expanded, and their children become the candidate set for the next stage. The process continues until leaf nodes are reached. A final leaf-level ranking then selects the passages supplied to the generator.

For a node set C_l at level l , the candidate score is

$$S_l(v \mid q) = s(q, v) + \lambda R(v, q), \quad (3)$$

where R is an optional routing term and λ controls its contribution. In the simplest implementation $R = 0$, so the hierarchy affects retrieval only through candidate restriction. A diversity constraint can then limit the number of selected leaves from the same source document, reducing redundant context.

4.3 Retrieval Algorithm

At query time, HAT (1) encodes and normalizes the query, (2) scores upper-level nodes, (3) expands the highest-scoring nodes under a beam width B , (4) repeats expansion toward leaves, and (5) reranks leaf passages under a document-diversity constraint before passing provenance-linked evidence to the generator. A flat-retrieval fallback is activated when hierarchical candidates fall below a confidence threshold. This fallback is important because it makes routing errors observable rather than allowing the hierarchy to silently suppress relevant evidence.

4.4 Cross-Document Evidence Aggregation

Cross-document questions often require evidence from more than one source. HAT therefore maintains document identifiers through every tree level and performs a final aggregation over the selected leaves. Let L_q be the ranked leaf set. The evidence selector can optimize

$$\max_{E \subseteq L_q} \sum_{v \in E} S(v \mid q) - \gamma \text{Redundancy}(E), \quad (4)$$

subject to a context budget. The redundancy term discourages near-duplicate passages, while the budget ensures that the final prompt remains compact.

Level	Representation	Retrieval role
Leaf	Source pas- sage + prove- nance	Final evidence
Intermediate	Cluster ab- straction	Semantic rout- ing
Upper	Cross- document abstraction	Topic naviga- tion
Root	Corpus ab- straction	Global query orientation

Table 1: Roles of the HAT hierarchy levels.

5 CUDA-Accelerated Retrieval Architecture

5.1 Execution Model

The proposed GPU mapping is based on the observation that embedding matrices and similarity computations are regular, dense numerical workloads. Document embeddings can be stored in contiguous device memory, and a batch of query or intermediate-node embeddings can be processed through matrix multiplication. Top- k selection can then be performed using GPU-friendly selection primitives. These operations are consistent with prior GPU similarity-search systems [Johnson et al. 2017] and dense retrieval methods that optimize the quality of approximate nearest-neighbor candidates [Xiong et al. 2021].

5.2 GPU Kernel Design Considerations

The principal kernels are (i) embedding-vector normalization, (ii) batched inner-product or cosine-similarity computation, and (iii) top- k candidate selection. Matrix-based similarity is naturally parallel because each query can be evaluated against many stored vectors. The implementation should minimize repeated transfers between host and device memory and should batch candidate groups to amortize kernel-launch overhead. IO-aware GPU design is relevant because memory traffic can become a limiting factor even when arithmetic throughput is high [Dao et al. 2022].

A practical implementation may use mixed precision for embeddings if retrieval quality remains stable, while retaining higher precision for score accumulation when necessary. The exact CUDA kernel configuration is hardware dependent and must therefore be reported experimentally rather than assumed. The CUDA programming guide provides the underlying execution and memory model for such an implementation [NVIDIA 2026].

5.3 Memory, Kernel, and Synchronization Strategy

Embeddings should be stored in contiguous device buffers, with normalization, batched similarity, and top- k selection kept on the GPU whenever possible. Candidate identifiers and provenance metadata should be transferred only when required by control logic. Batching, CUDA streams, pinned host memory, and mixed-precision storage may reduce memory pressure and launch overhead, but each optimization must be evaluated against retrieval quality. These choices make the CUDA contribution measurable rather than treating “GPU acceleration” as an unspecified implementation detail.

5.4 Complexity and Scaling

Suppose a flat index contains N leaf embeddings of dimension d . A brute-force query requires approximately $O(Nd)$ similarity work. With a hierarchy, the query evaluates a much smaller candidate set at each level. If b_l candidates are expanded at level l and the hierarchy has depth L , the search work is approximately

$$O\left(d \sum_{l=0}^L b_l\right), \quad (5)$$

plus the cost of final leaf reranking. The actual speed depends on how many nodes are expanded and on index implementation. Thus, HAT should be understood as a strategy for reducing the effective candidate set, not as a universal asymptotic guarantee.

6 Experimental Evaluation

The experimental study evaluates the proposed HAT architecture from three complementary perspectives: cross-document retrieval quality, evidence-grounded generation, and retrieval-system efficiency. All compared methods use the same document collection, text segmentation policy, embedding model, generator, context budget, and query set. This controls for factors unrelated to the retrieval architecture.

6.1 Research Questions and Hypotheses

The study addresses five research questions. **RQ1** asks whether hierarchical routing improves multi-document evidence recall at a fixed candidate budget. **RQ2** examines whether improved retrieval coverage transfers to answer grounding and source attribution. **RQ3** investigates how hierarchy depth, branching factor, and beam width affect the accuracy-latency trade-off. **RQ4** measures the contribution of CUDA acceleration to dense similarity computation and candidate selection. **RQ5** examines how the proposed approach behaves as corpus size and query batch size increase. The corresponding hypotheses are that HAT improves multi-source evidence coverage, maintains or improves grounding quality, and provides increasing computational benefits as dense retrieval workloads become larger.

6.2 Experimental Conditions and Baselines

The comparison includes flat dense retrieval, HNSW-based approximate nearest-neighbor retrieval, GPU vector retrieval without hierarchical routing, and a RAPTOR-inspired hierarchical retrieval configuration. HAT is evaluated as the proposed combination of cross-document abstraction, provenance-preserving routing, leaf reranking, and CUDA-accelerated dense retrieval. Identical embedding and generation settings are maintained across all configurations. The primary comparison uses matched candidate budgets so that a hierarchical method is not advantaged simply by examining more passages.

6.3 Dataset and Cross-Document Tasks

The evaluation corpus is organized into multiple documents covering related topics so that individual questions can require evidence from one or several sources. Three task categories are considered: single-source questions, multi-source conjunctive questions requiring complementary evidence, and multi-source comparative questions requiring the relationship between information from different documents. Each question is associated with a reference answer and evidence passages. Document-level separation is used between evaluation partitions to reduce leakage from near-duplicate passages.

6.4 Metrics

Retrieval performance is measured using Recall@ k , Precision@ k , MRR, and nDCG. A dedicated multi-source recall metric measures the proportion of required source documents represented among the retrieved evidence. Generation quality is assessed using answer relevance, faithfulness, claim-level support, and source attribution. ARES and RAGChecker provide useful evaluation dimensions for RAG systems [Saad-Falcon et al. 2024, Ru et al. 2024]. Efficiency is evaluated using retrieval latency, throughput, peak GPU memory, CPU memory, and index-construction time. Both average and tail latency are recorded because GPU systems may exhibit different behavior at high batch sizes.

6.5 Ablation and Reproducibility

The ablation study removes abstraction nodes, cross-document grouping, leaf reranking, CUDA execution, and the flat fallback independently. Additional experiments vary tree depth, branching factor, beam width, and leaf size. These comparisons separate the contribution of semantic hierarchy from that of GPU execution. Each run records the corpus version, chunking policy, embedding and generation models, vector dimensionality, tree parameters, numerical precision, batch size, GPU model, CUDA version, software versions, random seed, and index-construction time. Retrieval and generation metrics are reported over the same query set for all methods, and repeated runs are used for confidence intervals and paired statistical tests.

6.6 Expected Experimental Interpretation

The central evaluation criterion is not simply whether HAT retrieves highly similar passages, but whether it reliably brings complementary evidence from different documents into the final context while maintaining acceptable computational cost. A positive result would be demonstrated by improved multi-source recall at comparable candidate budgets, equal or better grounding quality, and a measurable reduction in dense-retrieval latency or increase in throughput on sufficiently large workloads. A negative result would be equally informative if hierarchy reduces recall, increases preprocessing cost without compensating retrieval gains, or provides no meaningful GPU benefit. This evaluation structure therefore makes the proposed contribution falsifiable rather than assuming that hierarchical organization is universally superior.

7 Discussion

HAT treats retrieval as navigation followed by evidence selection. A cross-document question may have a weak direct similarity signal at the leaf level but a stronger signal at a broader semantic level. Internal abstractions route the query toward relevant regions, while leaf passages remain the grounding source.

The novelty is intentionally bounded: RAPTOR already establishes tree-organized retrieval and GPU similarity-search systems already establish parallel vector search [Sarathi et al. 2024, Johnson et al. 2017]. HAT integrates these ideas around explicit cross-document grouping, provenance-preserving leaf selection, and CUDA-aware execution. Tree construction adds preprocessing cost, while GPU benefit is workload dependent because hierarchy traversal and metadata operations are less regular than dense similarity.

The main validity risks are confounding, abstraction quality, benchmark leakage, and hardware dependence; these are controlled through matched encoders, generators, hardware reporting, and evidence-level inspection.

8 Conclusion and Future Work

This paper proposed a Hierarchical Abstract Tree for cross-document retrieval-augmented generation and described how its computationally intensive retrieval stages can be mapped to NVIDIA CUDA-enabled GPU execution. HAT organizes source passages and higher-level abstractions in a provenance-preserving tree, enabling coarse-to-fine navigation before final leaf-level evidence selection. The architecture is motivated by limitations of flat retrieval and long-context generation and is grounded in prior work on RAG, dense retrieval, hierarchical retrieval, approximate nearest-neighbor search, and GPU similarity computation.

The evaluation framework provides a direct path to empirical validation of retrieval quality, grounding, latency, throughput, and GPU memory behavior. Adaptive hierarchy depth, incremental updates, multi-GPU indexing, and learned routing are promising extensions.

References

[Lewis et al. 2020] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks; In: Advances in Neural Information Processing Systems 33 (2020).

- [Guu et al. 2020] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.-W.: REALM: Retrieval-Augmented Language Model Pre-Training; In: Proc. ICML, PMLR 119, 3929–3938 (2020).
- [Karpukhin et al. 2020] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.-t.: Dense Passage Retrieval for Open-Domain Question Answering; In: Proc. EMNLP, 6769–6781 (2020).18653/v1/2020.emnlp-main.550.
- [Khattab and Zaharia 2020] Khattab, O., Zaharia, M.: ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT; In: Proc. 43rd International ACM SIGIR Conference, 39–48 (2020).1145/3397271.3401075.
- [Beltagy et al. 2020] Beltagy, I., Peters, M.E., Cohan, A.: Longformer: The Long-Document Transformer; arXiv:2004.05150 (2020).48550/arXiv.2004.05150.
- [Izacard and Grave 2021] Izacard, G., Grave, E.: Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering; In: Proc. EACL, 874–880 (2021).18653/v1/2021.eacl-main.74.
- [Liu et al. 2024] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the Middle: How Language Models Use Long Contexts; Transactions of the Association for Computational Linguistics 12, 157–173 (2024).1162/tacl_a_00638.
- [Li et al. 2024] Li, Z., Li, C., Zhang, M., Mei, Q., Bendersky, M.: Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach; In: Proc. EMNLP Industry Track, 881–893 (2024).18653/v1/2024.emnlp-industry.66.
- [Sarthi et al. 2024] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval; arXiv:2401.18059 (2024).48550/arXiv.2401.18059.
- [Malkov and Yashunin 2020] Malkov, Y.A., Yashunin, D.A.: Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs; IEEE Transactions on Pattern Analysis and Machine Intelligence 42(4), 824–836 (2020).1109/TPAMI.2018.2889473.
- [Johnson et al. 2017] Johnson, J., Douze, M., Jégou, H.: Billion-Scale Similarity Search with GPUs; arXiv:1702.08734 (2017).48550/arXiv.1702.08734.
- [Dao et al. 2022] Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Ré, C.: FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness; In: Advances in Neural Information Processing Systems 35 (2022).
- [Saad-Falcon et al. 2024] Saad-Falcon, J., Khattab, O., Potts, C., Zaharia, M.: ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems; In: Proc. NAACL-HLT, 338–354 (2024).18653/v1/2024.naacl-long.20.
- [Ru et al. 2024] Ru, D., Qiu, L., Hu, X., Zhang, T., Shi, P., Chang, S., Jiayang, C., Wang, C., Sun, S., Li, H., Zhang, Z., Wang, B., Jiang, J., He, T., Wang, Z., Liu, P., Zhang, Y., Zhang, Z.: RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation; In: Advances in Neural Information Processing Systems 37 (2024).52202/079017-0692.
- [Reimers and Gurevych 2019] Reimers, N., Gurevych, I.: “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”; Proc. EMNLP-IJCNLP, 3982–3992 (2019).
- [Xiong et al. 2021] Xiong, L., Xiong, C., Li, Y., Tang, K.-F., Liu, J., Bennett, P.N., Ahmed, J., Overwijk, A.: “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval”; Proc. International Conference on Learning Representations (2021).
- [NVIDIA 2026] NVIDIA Corporation: CUDA C++ Programming Guide; NVIDIA Developer Documentation (2026).